

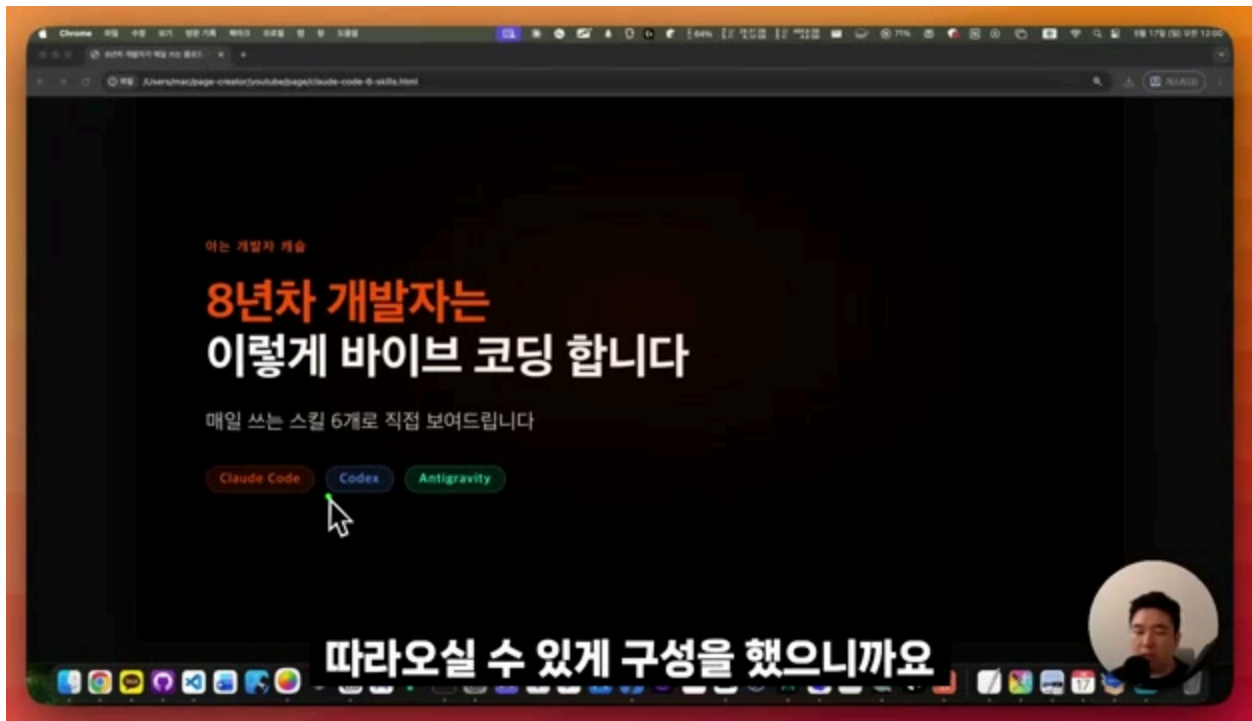
바이브코딩 핵심 스킬

바이브 코딩의 핵심은 무엇인가? 직접 프롬프트를 작성하기보다 **기존 스킬들을 최대한 활용**하여 효율적이고 퀄리티 높은 결과물을 얻는 것입니다.

1. 바이브 코딩의 핵심 스킬 6가지

바이브 코딩은 직접 프롬프트를 작성하기보다 기존 스킬들을 최대한 활용하여 효율적이고 퀄리티 높은 결과물을 얻는 것을 목표로 한다.

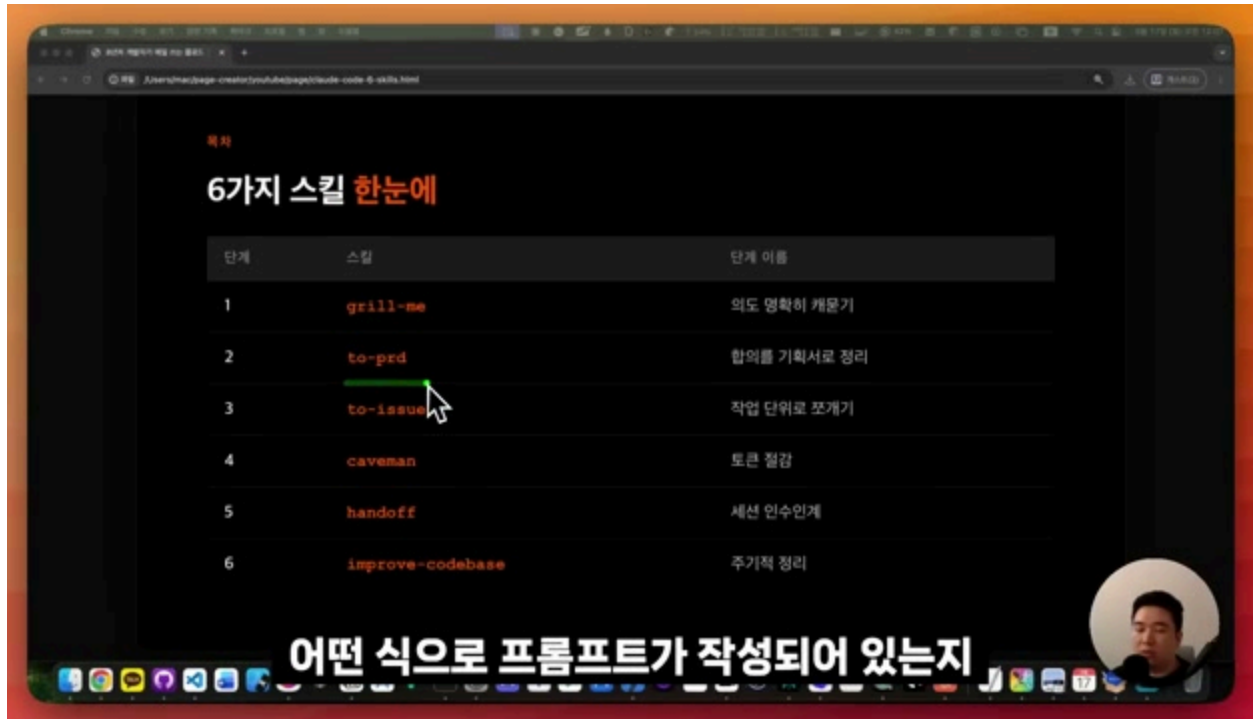
1.1. 바이브 코딩의 6가지 핵심 스킬



1. **Grill Me**: 대략적인 기획을 잡는 스킬이다.
2. **To PRD**: 문서를 만들고 구체적인 기획을 잡는 스킬이다.
3. **To Issues**: PRD 문서를 바탕으로 구체적인 작업 단위로 쪼개는 스킬이다.
4. **Caveman**: 작업 단위를 구현한 후 토큰을 절감하는 방법이다.
5. **Hand-off**: 작업 단위가 길어졌을 때 세션을 인수인계할 수 있게 하는 스킬이다.

6. **Improve Codebase**: 구현이 완료되었을 때 코드베이스를 개선하는 스킬이다.

1.2. Grill Me: 기획 초안 잡기



프롬프트를 일일이 작성하기 귀찮을 때,

Grill Me

스킬을 사용하면 AI가 필요한 질문을 스스로 하여 기획을 구체화한다.

1. 스킬의 작동 방식

- 'Grill'이라는 단어는 '꼬치꼬치 캐묻다', '취하다'라는 뜻을 가지며, 사용자에게 필요한 질문을 던져 아이디어를 구체화한다.
- 앱 개발 상황을 가정했을 때, 사용자가 만들고 싶은 앱과 기능을 설명하면 AI가 인터뷰 하듯 질문하여 정확한 요구사항을 파악한다.
- 이 스킬은 매트 포기라는 개발자가 만들었으며, 버셀 출신이다.

2. 실습 및 사용법

- VS Code에서 클로드 코드를 실행하고, `install` 명령어로 **Grill Me**와 **Grill Me Docs** 스킬을 설치한다.

- 설치 시 클로드 코드, 코덱스, 안티그래비티 중 사용할 AI를 선택하고, 프로젝트 레벨 또는 전체 프로젝트에서 사용할지 결정한다.
- 설치 후 `.agent` 폴더를 `.cloud` 폴더로 복사하여 스킬을 사용할 수 있도록 설정한다.
- `/grillme` 명령어로 스킬을 실행하고, "운동 기록 관리 웹을 만들고 싶어"와 같은 추상적인 프롬프트를 입력하면 AI가 구체적인 질문을 던진다.
- AI의 질문에 답변하면서 앱의 사용자, 데이터 입력 방식, 운동 종류 등을 구체화하며 최종 스펙을 완성한다.
- AI가 기술 스택까지 추천하며 바로 구현 가능한 수준의 스펙을 제공한다.

3. Grill Me Docs: 용어집 자동 정리

- **Grill Me**와 유사하게 인터뷰를 진행하지만, 대화 중 나오는 새로운 도메인 용어를 `context.md` 파일에 자동으로 정리한다.
- 운동 관리 앱 예시에서 '운동 종목', '세트', '기록'과 같은 용어를 일관되게 사용할 수 있도록 돕는다.

4. Grill Me 스킬 프롬프트 분석

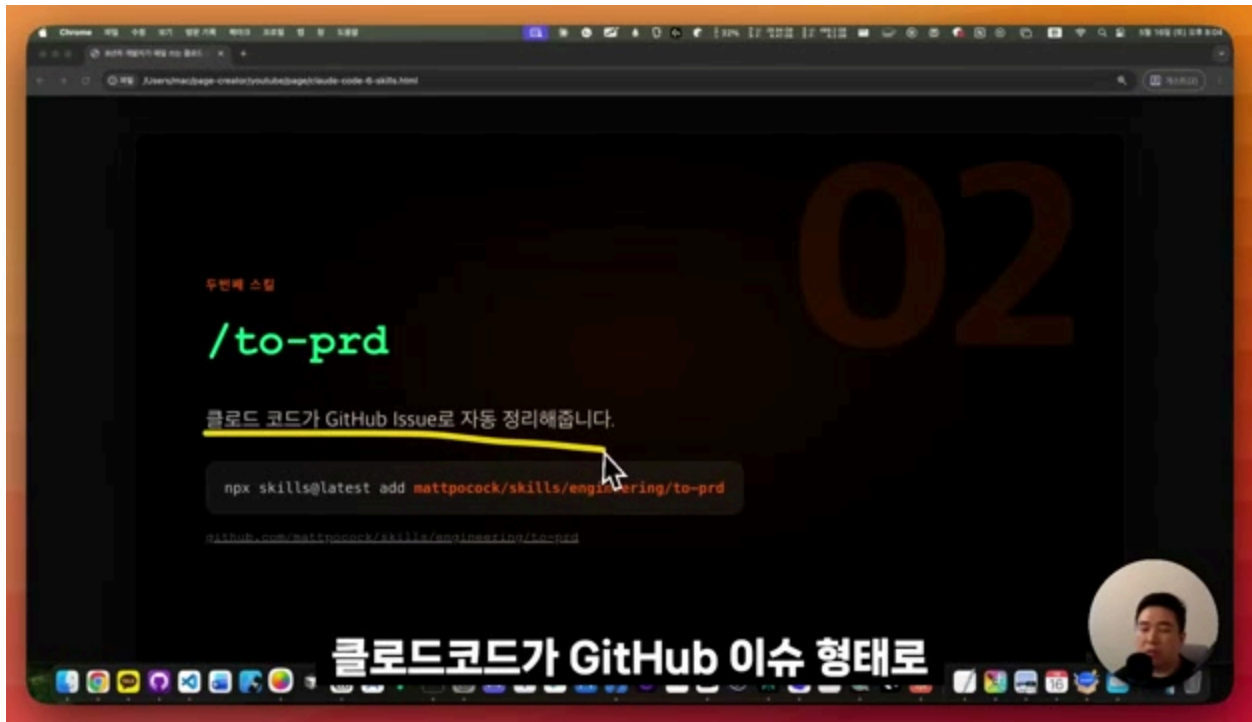
- 사용자가 계획이나 디자인에 대해 공통된 이해를 얻을 때까지 끈질기게 인터뷰하고, 의사 결정 트리의 각 분기점을 해결하도록 지시한다.
- 계획의 모든 측면에 대해 공통된 이해에 도달할 때까지 끊임없이 질문하고, 트리의 각 분기를 따라가며 결정 간의 의존 관계를 해결하도록 한다.
- 각 질문에 대한 권장 답변을 제시하고, 질문에 대한 답을 코드베이스에서 얻을 수 있다면 참고하도록 한다.
- 핵심은 '끈질기게 인터뷰', '권장 답변 제시', '한 번에 하나씩 질문', '코드베이스 참고'로 요약된다.

2. PRD 문서 자동 생성 및 이슈 트래커 연동

To PRD

스킬은 AI가 코드베이스를 파악하여 PRD(Product Requirements Document)를 자동으로 작성하고 이슈 트래커에 등록하는 기능이다.

2.1. To PRD 스킬 설치 및 실습



1. 설치 방법

- 새로운 터미널 창에서 설치 명령어를 실행하고, **To PRD** 스킬과 클로드 코드를 선택하여 설치한다.
- 프로젝트 레벨로 설치하고 권장 방식을 따른다.
- 설치 후 `.cloud` 및 `.agent` 폴더에 **To PRD** 폴더가 생성되어 클로드 코드, 코덱스, 안티그래비티에서 사용 가능하다.

2. 작동 방식 및 실습

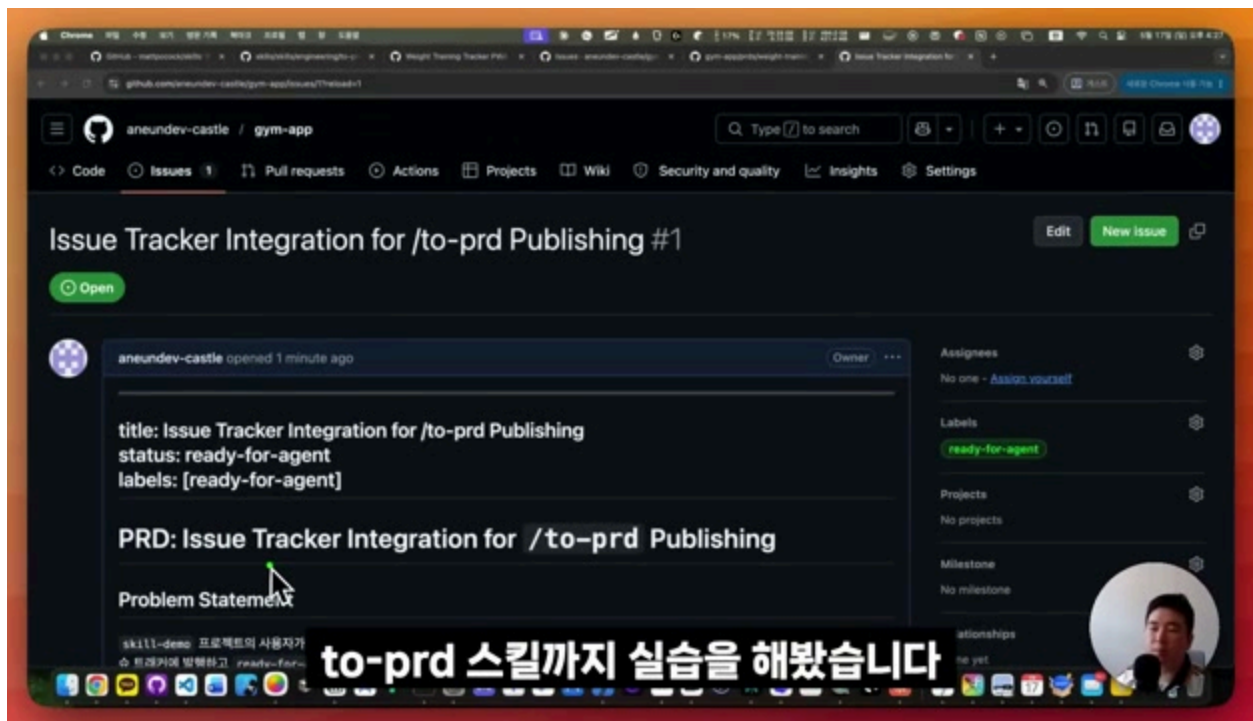
- **To PRD** 스킬은 대화 맥락과 코드베이스 이해를 바탕으로 PRD를 작성하고 이슈 트래커에 등록한다.
- 핵심 특징은 인터뷰 없이 코드베이스를 파악하여 이슈 트래커에 자동으로 등록하는 것이다.
- 이전 **Grill Me** 스킬로 대화가 끝난 기능을 바탕으로 PRD를 생성하고 이슈로 등록하는 도구로 활용된다.
- 스킬을 호출하여 "화해 주고 이슈로 등록해 줘"라고 요청하면 PRD가 생성된다.
- 이슈 트래커가 구성되어 있다면 해당 트래커에 등록되며, 그렇지 않으면 마크다운 파일로 정리된다.

- 생성된 PRD 문서는 **PRDs** 폴더에 저장되며, **PRD Issue Tracker Setup** 파일을 통해 GitHub 이슈 트래커에 등록할 수 있다.

3. GitHub 연동 및 PRD 이슈 등록

- 새로운 GitHub 레포지토리를 생성하고, 프로젝트를 해당 레포지토리에 업로드한다.
- 프로젝트를 GitHub에 푸시한 후, "PRD를 GitHub에 업로드 해줘"라고 요청한다.
- PRD가 GitHub에 업로드된 것을 확인하고, **PRD Issue Tracker Setup** 파일을 참고하여 GitHub 이슈에 등록하도록 요청한다.
- 성공적으로 등록된 PRD 문서를 링크를 통해 확인할 수 있다.

2.2. To PRD 스킬 프롬프트 분석



1. 스킬의 목적 및 기능

- 현재 대화 컨텍스트를 PRD로 변환하여 프로젝트 이슈 트래커에 게시하는 스킬이다.
- 사용자가 현재 컨텍스트에서 PRD를 생성하려는 경우에 사용된다.

2. 작동 흐름

- 코드베이스의 현재 상태를 파악하고, 프로젝트 도메인 용어집의 어휘를 사용한다.

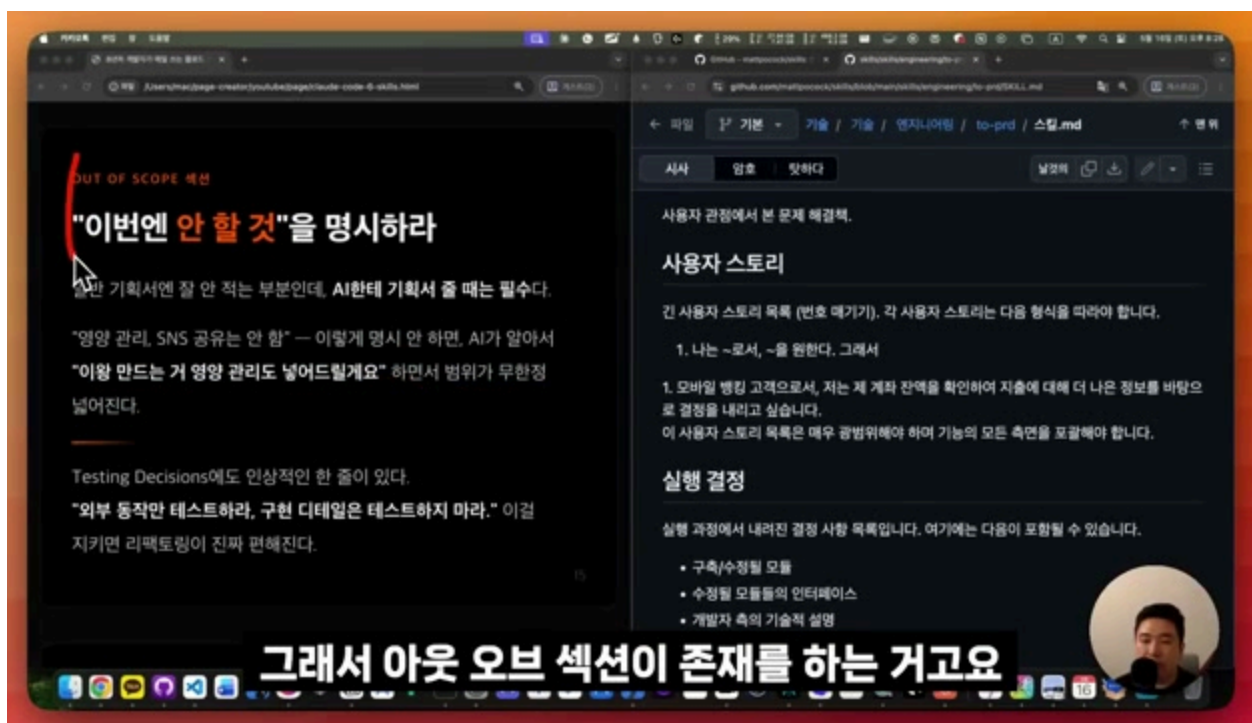
- 구현 완료에 필요한 주요 모듈을 파악하고 설명한다.
- 제시된 템플릿을 사용하여 PRD를 작성하고 프로젝트 이슈 트래커에 게시한다.

3. PRD 템플릿 분석

- 템플릿은 문제 정의, 해결 방안, 사용자 스토리 등으로 구성된다.
- **사용자 스토리**: 역할을 정의하고, 그 역할이 원하는 것과 그 이유를 명확히 작성하도록 강제한다. (예: 헬스하는 사용자가 지난번 무게를 자동 표시하는 것을 원한다. 왜냐하면 매번 기억할 필요가 없기 때문이다.)
- **아웃오브 스코프(Out of Scope)** 섹션: 이번 구현 범위에 포함되지 않는 기능을 명시하여 AI가 범위를 확장하는 것을 방지한다. (예: 영양 관리나 SNS 공유 기능은 넣지 않겠다.)

3. 작업 단위 분할 및 토큰 절감

3.1. To Issues: PRD를 작업 가능한 이슈로 분할



To Issues

스킬은 작성된 PRD 문서를 독립적이고 작업 가능한 이슈들로 쪼개어 AI가 순서대로 구현할 수 있도록 돕는다.

1. 스킬 설치 및 실습

- 이전과 동일한 방식으로 **To Issues** 스킬을 설치한다.
- 클로드 코드를 실행하고 `/toissues` 명령어로 스킬을 호출한다.
- "PRD 문서를 작업 가능한 단위로 쪼개고 GitHub 이슈로 등록해 줘"라고 요청한다.
- AI는 PRD를 9개의 수직 슬라이스로 쪼개고, GitHub 이슈 발행을 위해 명시적인 확인을 요청한다.
- 사용자의 확인 후, 9개의 이슈가 순차적으로 발행되며 GitHub에서 확인할 수 있다.
- 각 이슈는 구현해야 할 내용과 성공 기준, 선행 작업 등을 명확히 포함한다.

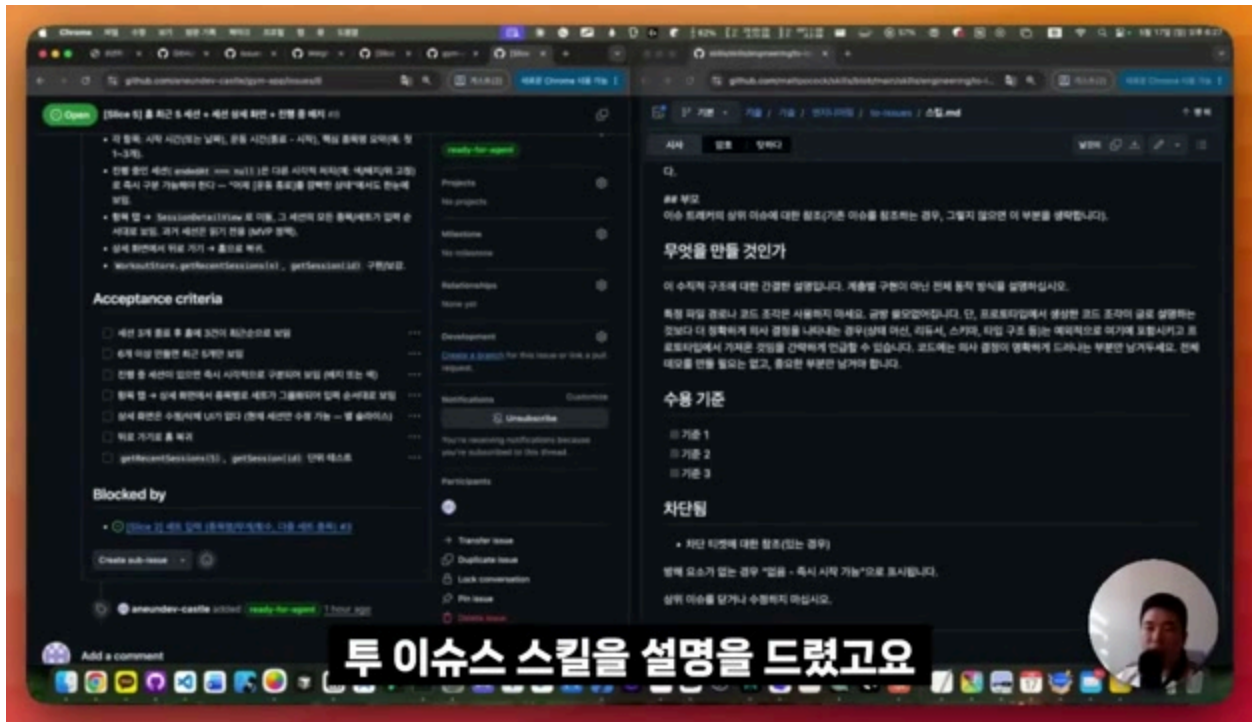
2. 수평 슬라이스와 수직 슬라이스 개념

- **수평 슬라이스**: DB, API, UI 등 각 레이어를 한 번에 모두 구현하는 방식이다.
- **수직 슬라이스**: 특정 기능(예: 운동 종목 추가)에 대해 DB, API, 화면, 테스트 등 관련 부분을 조금씩 구현하여 기능을 완성하는 방식이다.
- **To Issues** 스킬은 기능 단위로 쪼개는 수직 슬라이스 방식을 사용한다.

3. To Issues 스킬 프롬프트 분석

- 대화 맥락과 코드베이스를 바탕으로 상황을 파악하고, 이슈 트래커에서 해당 이슈를 가져와 본문과 댓글을 읽는다.
- 코드베이스를 살펴보고, 특정 계층 하나만을 수평으로 자르는 단면이 되지 않도록 수직 단면 초안 작성을 지시한다.
- 사용자에게 해상도 적절성, 슬라이스 병합 또는 추가 분할 필요성 등을 물어보며 분석 결과를 승인받을 때까지 반복한다.
- 승인된 각 슬라이스에 대해 이슈 트래커에 새 이슈를 게시하며, 템플릿에 따라 '무엇을 만드는지', '확인 기준', '완료되어야 할 작업'을 포함하도록 한다.
- 템플릿은 기능 단위로 이슈를 생성하고, 성공 기준과 선행 작업을 명확히 제시한다.

3.2. Caveman: 토큰 절감 및 간결한 답변



Caveman

스킬은 AI의 답변을 매우 간결하게 만들어 토큰 사용량을 줄이는 스킬이다.

1. 스킬 소개 및 설치

- 19살 대학생이 만든 스킬로, 글로벌 트렌딩 1위를 기록했다.
- 설치 명령어를 실행하여 **Caveman** 스킬을 설치한다.
- 설치 후 클로드 코드를 실행하여 `/caveman` 명령어로 스킬을 확인하고, `caveman help` 로 설명을 볼 수 있다.

2. 효과 및 실습

- **Caveman** 모드를 트리거하면 AI가 장황한 답변 대신 19개 토큰을 사용하며 매우 간결하게 답변한다.
- "이 프로젝트에서 개선할 점 모두 설명해 줘"와 같은 요청에 대해 불필요한 말 없이 핵심 내용만 전달한다.
- 이를 통해 토큰을 절감할 수 있다.

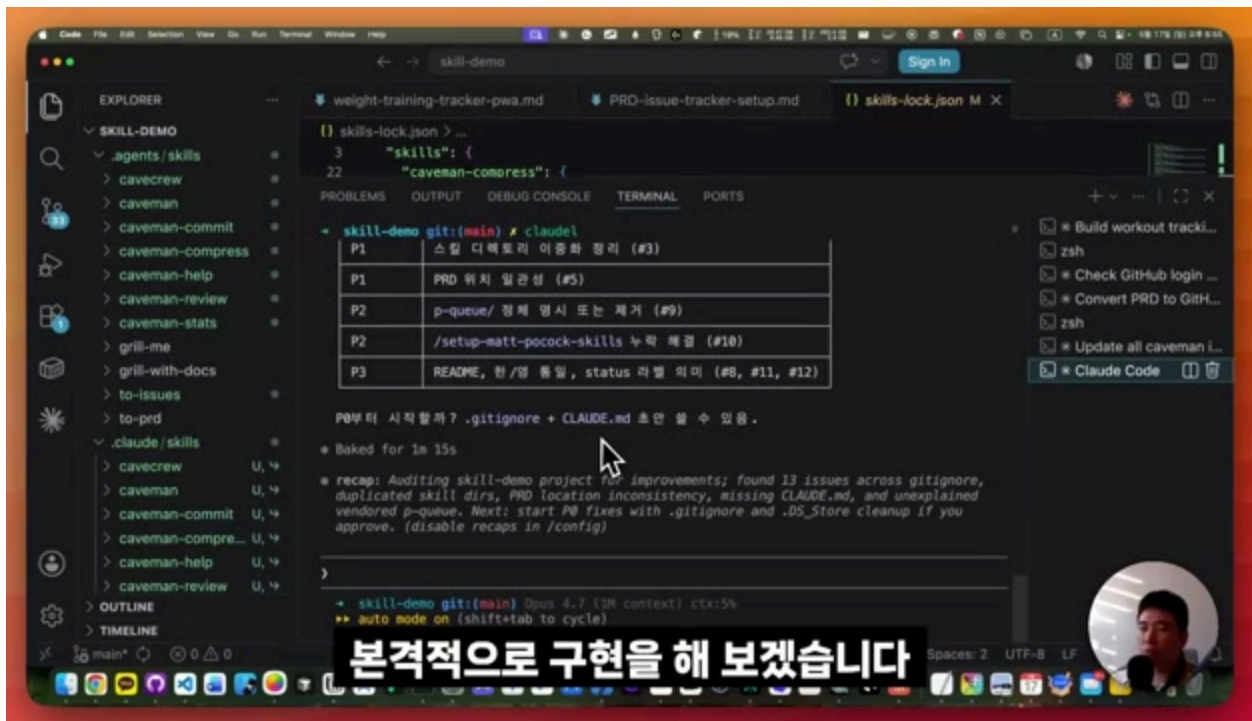
3. Caveman 스킬 프롬프트 분석

- "똑똑한 원신처럼 말하라"는 지시와 함께, 모든 응답을 산문으로 압축하도록 한다.

- 관사, 군더더기 표현, 경구, 완곡한 표현 등을 제거하고, 기술적 세부 사항이나 코드 블록은 정확하게 유지한다.
- 핵심은 불필요한 표현을 쓰지 않고 필요한 말만 정확하게 하는 것이다.

4. 구현 진행 및 코드 개선

4.1. 본격 구현 및 Hand-off: 세션 인수인계



To Issues

스킬로 분할된 작업 단위 순서대로 구현을 진행하며, 작업이 길어질 경우

Hand-off

스킬을 사용하여 세션 인수인계 문서를 작성한다.

1. 구현 진행

- GitHub 이슈에 등록된 슬라이스 1부터 순서대로 구현을 요청한다.
- AI는 GitHub 이슈를 읽어오며 구현을 진행한다.
- 슬라이스 2 구현 완료 후, 테스트 11개가 모두 통과되었음을 확인한다.

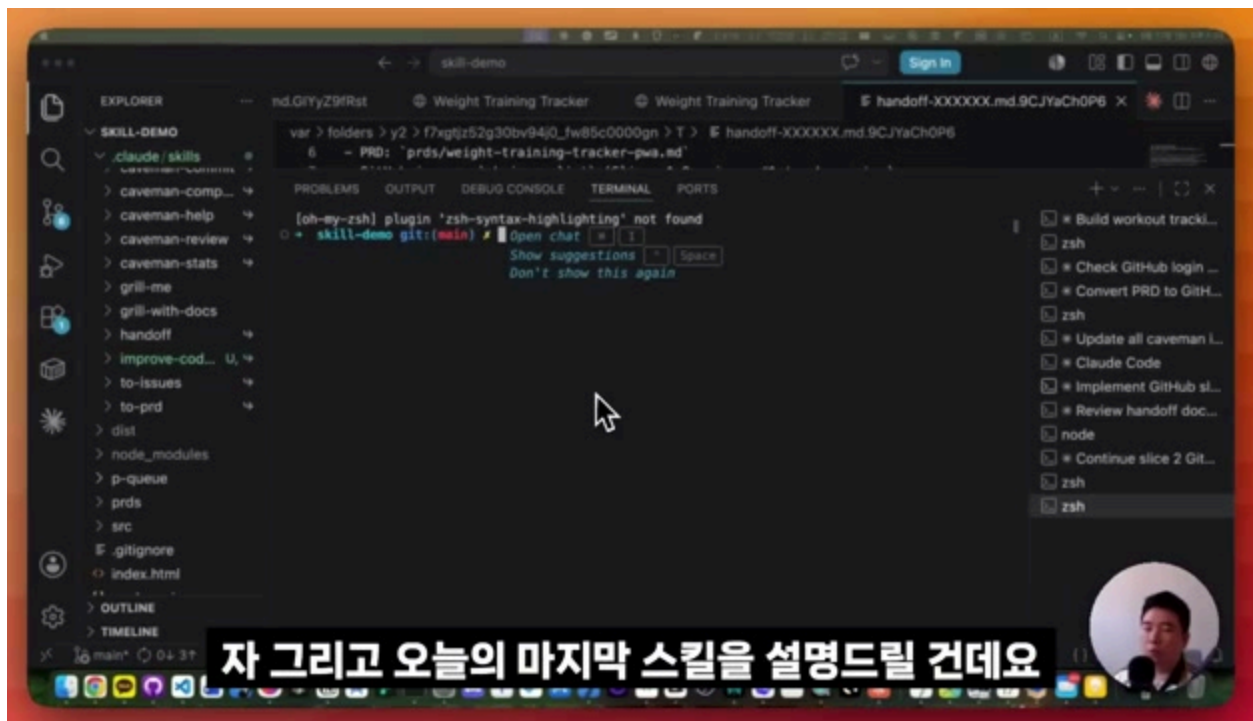
2. Hand-off 스킬 설치 및 실습

- **Hand-off** 스킬을 설치한다.
- 클로드 코드를 실행하고 `/handoff` 명령어로 스킬을 호출하여 인수인계 문서를 작성한다.
- 작성된 문서를 바탕으로 새로운 세션에서 슬라이스 2부터 작업을 이어나간다.
- 이 스킬은 결정 사항을 문서에 기록하고 내용을 명확히 하여, 새로운 세션에서 해당 문서를 참고해 구현을 시작할 수 있도록 돕는다.

3. 구현 결과 확인

- 앱을 실행하여 첫 번째 단계 구현 결과를 확인한다.
- 운동 시작, 종료 기능이 정상 작동하며 최근 운동 기록이 반영된다.
- 슬라이스 2 구현 후, 운동 종목, 무게, 횟수 입력 및 기록 등록 기능이 정상 작동함을 확인한다.

4.2. Improve Codebase: 코드베이스 개선



Improve Codebase

스킬은 코드베이스를 '깊은 모듈' 형태로 개선하여 품질을 향상시키는 스킬이다.

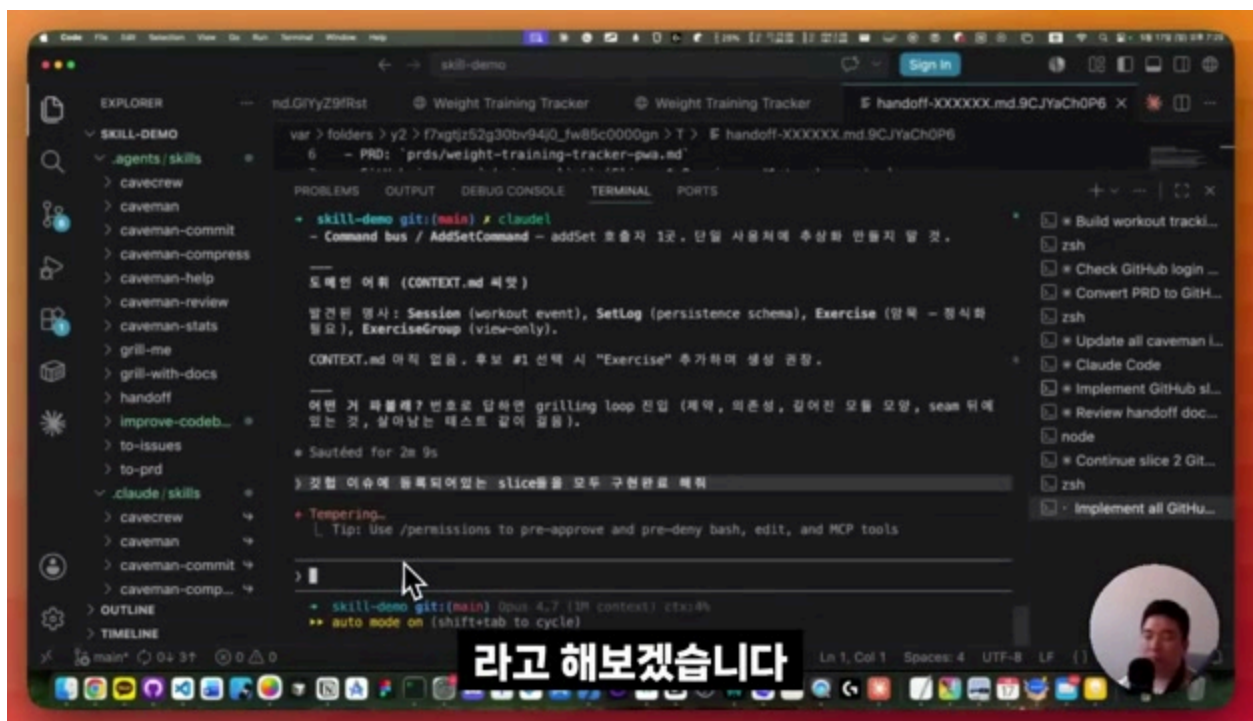
1. 스킬 설치 및 개념 설명

- **Improve Codebase Architecture** 스킬을 설치한다.
- 클로드 코드를 실행하고 `/improvecodebase` 명령어로 스킬을 호출하여 설명을 요청한다.
- **깊은 모듈**: 인터페이스는 단순하지만 내부 구현이 풍부한 모듈을 의미한다.
- **얕은 모듈**: 인터페이스가 다양하고 복잡하지만 내부 구현이 적은 모듈을 의미한다.
- 이 스킬은 코드베이스를 깊은 모듈로 바꿔 품질을 개선한다.

2. 스킬 적용

- "코드 베이스 개선 시작"을 요청하여 스킬을 적용한다.
- AI는 깊은 모듈로 바꿀 수 있는 후보들을 제시한다.
- 모든 슬라이스 구현 완료를 요청한다.

4.3. 최종 결과물 및 바이브 코딩 핵심 메시지



모든 작업이 완료된 후, GitHub 이슈가 모두 처리되고 최종 앱 결과물을 확인한다.

1. 최종 결과물 확인

- GitHub 이슈가 모두 완료 처리되고 댓글로 완료 내용이 기록된다.
- 완성된 앱에서 운동 기록 추가 및 상세 보기 기능이 정상 작동함을 확인한다.

2. 바이브 코딩의 핵심

- 바이브 코딩을 잘하는 사람은 직접 작성하는 프롬프트가 적다.
- 이미 잘 만들어진 스킬들을 최대한 활용하여 효율적이고 퀄리티 높은 결과물을 만들어 낸다.